Solving Generalized Sudoku by Simulated Annealing

Nguyen Bui
Department of Mathematics, DePauw University

Spring 2026
Professor McKenzie Lamb

Abstract

This paper studies simulated annealing as a stochastic optimization method for solving generalized Sudoku puzzles. A generalized Sudoku puzzle has size $n^2 \times n^2$, uses the symbols $1, \ldots, n^2$, and requires each row, column, and $n \times n$ subgrid to contain each symbol exactly once. The algorithm restricts the search space to boards that already satisfy all fixed clues and all subgrid constraints. Proposed moves swap two non-fixed entries inside one subgrid, so the subgrid constraints remain true throughout the search. The energy function therefore counts only repeated symbols in rows and columns, and a zero-energy state is exactly a valid solution. The paper compares greedy local search with simulated annealing, uniform proposals with violation-weighted proposals, and three recovery strategies for runs that terminate with positive energy: full restart, re-annealing, and hybrid recovery. The experiments also examine the effect of increasing the iteration budget and the largest board sizes attempted. The results show that simulated annealing provides a useful framework for generalized Sudoku, but exact solve rate alone is not a sufficient performance measure. Final energy, computation time, recovery attempts, and board size reveal additional information about how the algorithm behaves near local minima.

1 Introduction

Sudoku is a logic puzzle built from a simple set of constraints. In the standard 9×9 version, the board is divided into nine 3×3 subgrids, and the symbols $1, \ldots, 9$ must be placed so that no symbol repeats in any row, column, or subgrid. A puzzle begins with some fixed entries, called clues, and the goal is to fill the remaining cells without changing those clues. Although Sudoku is often presented as a recreational puzzle, it is also a finite constraint satisfaction problem: a completed board is valid exactly when it satisfies all row, column, subgrid, and clue constraints.

The same idea can be generalized beyond the usual 9×9 board. For a positive integer n, a generalized Sudoku puzzle has size $n^2 \times n^2$, uses the symbols $1, \ldots, n^2$, and is divided into $n \times n$ subgrids. The usual Sudoku puzzle corresponds to $n = 3$. When $n = 4$, the board becomes 16×16, and when $n = 5$, the board becomes 25×25. The rules are structurally the same, but the number of cells, constraints, and possible board completions grows quickly.

Generalized Sudoku is useful for this project because larger boards are much harder to solve by ordinary methods. On a standard 9×9 puzzle, deterministic backtracking, constraint propagation, and human-style solving rules can often find a solution efficiently. On 16×16 and 25×25 puzzles, the search space becomes much larger, and local decisions interact across many rows, columns, and subgrids. A move that appears helpful in one part of the board can create conflicts somewhere else. This makes the problem a good testing ground for stochastic search methods.

This difficulty leads naturally to Markov chain Monte Carlo ideas and simulated annealing. Instead of trying to construct the solution directly, the algorithm moves randomly through a carefully chosen space of completed boards. Each board has an energy value measuring how far it is from satisfying the Sudoku constraints. The algorithm prefers moves that lower the energy, but it sometimes accepts moves that raise the energy. This is important because a board can be close to solved while still trapped in a local minimum. Allowing occasional uphill moves gives the search a way to escape such traps.

Simulated annealing was introduced as a general optimization method by Kirkpatrick, Gelatt, and Vecchi [2], building on the Metropolis algorithm of statistical physics [1]. The method uses a temperature parameter to control the probability of accepting energy-increasing moves. At high temperature, the algorithm explores broadly. At low temperature, it behaves more like greedy descent. A cooling schedule gradually lowers the temperature, attempting to balance exploration and exploitation.

The purpose of this paper is twofold. First, the mathematical structure of the algorithm is described precisely: the state space, energy function, proposal rules, acceptance probability, and cooling schedule are all defined. Second, several algorithmic choices are compared experimentally. These comparisons include greedy local search versus simulated annealing, uniform proposals versus violation-weighted proposals, and multiple recovery strategies for runs that end with positive energy. The paper also addresses the effect of increasing the iteration budget, the difficulty of 25×25 boards, and the possibility of parallelization.

2 Mathematical Model and Algorithm

2.1 Definitions and Mathematical Setup

Let $m = n^2$. A generalized Sudoku board is an $m \times m$ array with entries in the set $\{1, \dots, m\}$. The board is divided into m subgrids, each of size $n \times n$. A valid completed board satisfies three conditions:

1. Each row is a permutation of $\{1, \dots, m\}$.
2. Each column is a permutation of $\{1, \dots, m\}$.
3. Each $n \times n$ subgrid is a permutation of $\{1, \dots, m\}$.

A puzzle instance also contains fixed clue cells. These entries cannot be changed. Therefore a solution is not merely any valid completed board; it must also agree with all given clues.

The implementation uses a restricted state space in order to reduce the number of constraints that must be handled during the search. Let $\mathcal{X}$ denote the set of boards satisfying two conditions:

1. All fixed clues are preserved.
2. Each $n \times n$ subgrid is a valid permutation of $\{1, \dots, m\}$.

This means that every board in $\mathcal{X}$ already satisfies the clue constraints and subgrid constraints. The only remaining constraints that can be violated are row and column constraints. The search problem is therefore transformed into a row-column repair problem.

For a board $X \in \mathcal{X}$, define the energy function

$$\mathcal{E}(X) = \sum_{i=1}^{m} (m - d_i^{\text{row}}(X)) + \sum_{j=1}^{m} \left(m - d_j^{\text{col}}(X)\right),$$

where $d_i^{\text{row}}(X)$ is the number of distinct symbols in row i, and $d_j^{\text{col}}(X)$ is the number of distinct symbols in column j. A valid row has m distinct symbols and contributes 0 to the energy. A row with repeated symbols contributes a positive penalty. Columns are treated in the same way.

Because all boards in $\mathcal{X}$ already satisfy the clue and subgrid constraints, the condition $\mathcal{E}(X) = 0$ is equivalent to X being a valid Sudoku solution. Thus the problem becomes an optimization problem: find a state $X \in \mathcal{X}$ minimizing $\mathcal{E}(X)$, ideally reaching energy zero.

This energy function is not a perfect distance to the nearest solution. Two boards can have the same energy but different levels of difficulty. However, it is a useful performance measure. A board with energy 2 is much closer to a solution than a board with energy 100, even though neither is a valid solution. This distinction becomes important in the results, because several methods fail to solve exactly but still end at different energy levels.

2.2 Initialization and Move Structure

The algorithm begins in the restricted state space $\mathcal{X}$ and never leaves $\mathcal{X}$. Initialization is performed block by block. In each subgrid, the fixed clue values are recorded, and the missing symbols are randomly assigned to the non-fixed cells in that subgrid. Therefore each subgrid is valid immediately after initialization.

The move structure also preserves $\mathcal{X}$. A proposed move consists of selecting two non-fixed cells in the same subgrid and swapping their values. Since the swap occurs inside a single subgrid, the multiset of symbols in that subgrid does not change. Since both cells are non-fixed, no clue is changed. Thus every proposed board remains in $\mathcal{X}$.

This restriction is a major modeling choice. It removes the need to penalize subgrid violations, because such violations are impossible. The algorithm only needs to remove row and column repetitions while maintaining the already-satisfied block constraints.

The move structure also gives a computational advantage. If two cells (r_1, c_1) and (r_2, c_2) are swapped, only rows r_1, r_2 and columns c_1, c_2 can change their violation counts. All other rows and columns remain unchanged. Therefore the implementation computes the energy difference locally instead of recomputing the full board energy after every proposed move.

2.3 Greedy Local Search

Greedy local search uses the same state space, energy function, and block-preserving swap proposals as simulated annealing. The only difference is the acceptance rule. If a proposed swap has $\Delta\mathcal{E} \leq 0$, the greedy algorithm accepts it; if $\Delta\mathcal{E} > 0$, the move is rejected. Thus greedy search never intentionally makes the board worse.

This method is included because it isolates the value of uphill moves. Simulated annealing and greedy search both operate on the same restricted Sudoku state space, but simulated annealing can escape local minima by accepting some energy-increasing swaps. The comparison with greedy search is therefore conceptually important because it tests whether the main advantage of simulated annealing actually matters for generalized Sudoku.

2.4 Simulated Annealing

At a fixed temperature $T > 0$, the uniform-proposal version of the algorithm defines a Metropolis chain on the finite state space $\mathcal{X}$. From a current board X, a candidate board Y is proposed by swapping two non-fixed cells inside one subgrid. Let

$$\Delta = \mathcal{E}(Y) - \mathcal{E}(X).$$

The move is accepted with probability

$$a(X, Y) = \min\{1, e^{-\Delta/T}\}.$$

If $\Delta \leq 0$, the proposed move is accepted. If $\Delta > 0$, the move is accepted with probability $e^{-\Delta/T}$. Larger increases in energy are less likely to be accepted, and all uphill moves become less likely as T decreases.

For a symmetric proposal rule at fixed temperature, the Metropolis chain has stationary distribution proportional to

$$\pi_T(X) \propto e^{-\mathcal{E}(X)/T}.$$

This explains why the method favors low-energy states. At smaller temperatures, the distribution gives more weight to states with lower energy.

However, the full simulated annealing algorithm is not a fixed-temperature chain. The temperature changes over time, so the transition probabilities also change over time. The process is time-inhomogeneous. The fixed-temperature stationary distribution explains the intuition behind the acceptance rule, but it is not by itself a convergence proof for the full cooling schedule.

This distinction matters because the Fundamental Theorem for Markov chains applies to a fixed transition matrix. Simulated annealing uses a sequence of transition matrices, one for each temperature. Classical convergence results require additional assumptions on the cooling schedule; for example, logarithmic cooling can be theoretically sufficient in certain finite-state settings [3]. In practice, logarithmic cooling is often too slow. The implementation therefore uses a geometric cooling schedule,

$$T_k = T_0 \alpha^k,$$

where $0 < \alpha < 1$.

The initial temperature T_0 is estimated using trial moves. The algorithm samples random swaps and records the positive energy changes. If $\overline{\Delta\mathcal{E}_+}$ is the average positive change, then

$$T_0 = -\frac{\overline{\Delta\mathcal{E}_+}}{\log(p_0)},$$

with $p_0 = 0.80$. This means that at the beginning of the run, an average uphill move is accepted with probability approximately 80%. This heuristic makes the starting temperature adapt to the scale of the current puzzle.

2.5 Proposal Rules

Two proposal rules are compared. The first is uniform. A subgrid with at least two non-fixed cells is chosen, and then two such cells in that subgrid are chosen uniformly at random. This proposal is symmetric: if one swap can take X to Y, the reverse swap can take Y back to X. Therefore the standard Metropolis acceptance probability is appropriate.

The second proposal rule is violation-weighted. The goal is to guide the algorithm toward parts of the board that are involved in conflicts. For each subgrid B, define

$$w(B) = 1 + \sum_{r \in R(B)} v_r + \sum_{c \in C(B)} v_c,$$

where $R(B)$ is the set of rows passing through B, $C(B)$ is the set of columns passing through B, and v_r, v_c are row and column violation counts. The subgrid is selected with probability proportional to $w(B)$. The added 1 ensures that every movable block can still be selected.

Inside the selected block, cells are also weighted. A non-fixed cell in row r and column c receives weight $1 + v_r + v_c$. Thus the proposal is more likely to swap cells participating in row or column conflicts.

This weighted proposal is a heuristic. Since the proposal probability depends on the current state, the proposal is not generally symmetric. A fully correct Metropolis-Hastings version would use

$$\min\left\{1, \exp\left(-\frac{\mathcal{E}(Y) - \mathcal{E}(X)}{T}\right)\frac{q(Y,X)}{q(X,Y)}\right\}.$$

The implemented weighted method does not include this correction, so it should be interpreted as a guided search heuristic rather than a theoretically exact Metropolis chain.

2.6 Recovery Strategies

When a run ends with $\mathcal{E}(X_{\text{best}}) = 0$, the puzzle is solved. When it ends with $\mathcal{E}(X_{\text{best}}) > 0$, the returned board is not a valid solution. This case is important because many failed runs end near a solution.

The first recovery strategy is full restart. The current board is discarded, a new random initialization is created, and annealing begins again. Restart can escape a poor region of the state space, but it also discards useful progress.

The second strategy is re-annealing. The algorithm keeps the current best board but resets the temperature to a moderate value. This allows uphill moves again while preserving the partial structure already found. Re-annealing is especially useful when the current board has low positive energy.

The third strategy is hybrid recovery. Hybrid recovery re-anneals a few times from the current board, but if those attempts fail, it restarts. This combines the local preservation of re-annealing with the global escape of restart. Conceptually, hybrid recovery should be the strongest recovery strategy because it can use both forms of recovery.

3 Experimental Design and Results

3.1 Experimental Design

The experiments were designed to test four main factors: proposal rule, recovery strategy, iteration budget, and board size. These factors were chosen because they correspond directly to the main algorithmic questions in the project. The proposal rule determines how the algorithm chooses a neighboring board. The recovery strategy determines what happens when a run terminates with positive energy. The iteration budget controls how long the algorithm is allowed to search. The board size measures how well the method scales from ordinary Sudoku to generalized Sudoku.

All experiments used the same basic Sudoku representation and energy function described in Section 2. The search space was restricted to boards that preserve the fixed clues and satisfy all subgrid constraints. Therefore the energy function counted only row and column violations. Each run began by randomly filling the non-fixed cells inside each subgrid so that every subgrid was already a permutation of the required symbols. From that point onward, every proposed move swapped two non-fixed entries inside the same subgrid. This ensured that all methods were compared on the same constrained search space.

The main comparison was performed on 16×16 Sudoku boards. This size was chosen because it is large enough to be meaningfully harder than ordinary 9×9 Sudoku, but still small enough that many trials could be run in a reasonable amount of time. A 9×9 puzzle is often too easy to

show the difference between greedy search, uniform simulated annealing, and weighted simulated annealing. A 25×25 puzzle is much harder and can require much longer computation. The 16×16 case therefore served as the primary test case for comparing algorithmic choices.

The first experiment compared three baseline search methods on 16×16 puzzles: greedy local search, uniform simulated annealing, and weighted simulated annealing. This comparison was included to isolate the effect of uphill moves and proposal guidance. Greedy search accepts only non-worsening moves, so it tests what happens when the algorithm has no mechanism for escaping local minima. Uniform simulated annealing uses the same move structure but sometimes accepts uphill moves, so it tests the value of the Metropolis acceptance rule. Weighted simulated annealing adds conflict-directed proposals, so it tests whether using violation information improves the search beyond ordinary simulated annealing.

The second experiment compared combinations of proposal rules and recovery strategies on 16×16 puzzles. The proposal rules were uniform and violation-weighted. The recovery strategies were no recovery, full restart, re-annealing, and hybrid recovery. This produced eight combinations:

uniform-none, uniform-restart, uniform-reanneal, uniform-hybrid,
weighted-none, weighted-restart, weighted-reanneal, weighted-hybrid.

This setup makes the experiment more informative than comparing only one version of simulated annealing against one baseline. By crossing proposal rule with recovery strategy, the results show whether improvement comes mainly from better proposals, better recovery after failure, or the combination of both.

The recovery comparison was especially important because simulated annealing often terminates with $E(X_{\text{best}}) > 0$. In that case, the algorithm has not found a valid solution, but the best board may still be close to valid. The no-recovery condition measures what happens if the algorithm simply stops. The restart condition tests whether it is better to discard the current board and begin again from a new random initialization. The re-annealing condition tests whether it is better to keep the current board and raise the temperature so that the search can escape a local minimum. The hybrid condition tests a combined strategy: re-anneal first, and restart only if repeated re-annealing does not solve the puzzle. This design makes it possible to compare different responses to positive-energy termination.

The third experiment studied the effect of iteration budget on 16×16 puzzles. The weighted hybrid method was tested with several budgets, including 50,000, 100,000, 200,000, and 500,000 iterations. This experiment was included because a simulated annealing algorithm can perform very differently depending on how long it is allowed to run. A small budget may stop before the search has time to escape early local minima. A larger budget gives the algorithm more opportunities to accept useful uphill moves and later cool into a low-energy region. However, a larger budget also increases computation time, and if the cooling schedule is not tuned well, extra iterations may not always improve performance. For that reason, the budget experiment evaluates both solve rate and final energy.

The fourth experiment studied board size. The weighted hybrid method was tested on 9×9, 16×16, and 25×25 puzzles. This experiment was included to answer how large a puzzle the algorithm could solve under the tested settings. The 9×9 case serves as a standard Sudoku reference point. The 16×16 case is the main generalized Sudoku test case. The 25×25 case tests the limits of the implementation and shows whether the same algorithmic choices still work when the search space becomes much larger.

For each experiment, the main recorded quantities were solve rate, mean final energy, and mean computation time. Solve rate is the percentage of runs that ended with energy zero. Mean final energy measures the average number of remaining row and column violations at termination.

Computation time measures the practical cost of each method. These three quantities were reported together because none of them alone gives a complete description of performance. Solve rate tells whether the puzzle was exactly solved, final energy tells how close failed runs came to a solution, and computation time tells how expensive the method was.

The plotted results should be interpreted descriptively rather than as formal hypothesis tests. Simulated annealing is random, so a larger number of trials would be needed for strong statistical conclusions. The purpose of these experiments is to identify meaningful performance trends and to compare algorithmic choices in a controlled way. The results are therefore analyzed through observed differences in solve rate, final energy, and runtime rather than through formal p-values.

3.2 Baseline Comparison

The first experimental comparison evaluates greedy local search against simulated annealing. This baseline is important because it separates the effect of accepting uphill moves from the effect of using a guided proposal rule. Greedy local search accepts only swaps that do not increase the energy. Uniform simulated annealing uses the same type of block-preserving swap, but it sometimes accepts energy-increasing moves according to the Metropolis probability. Weighted simulated annealing adds a second improvement by directing proposals toward rows and columns with more violations.

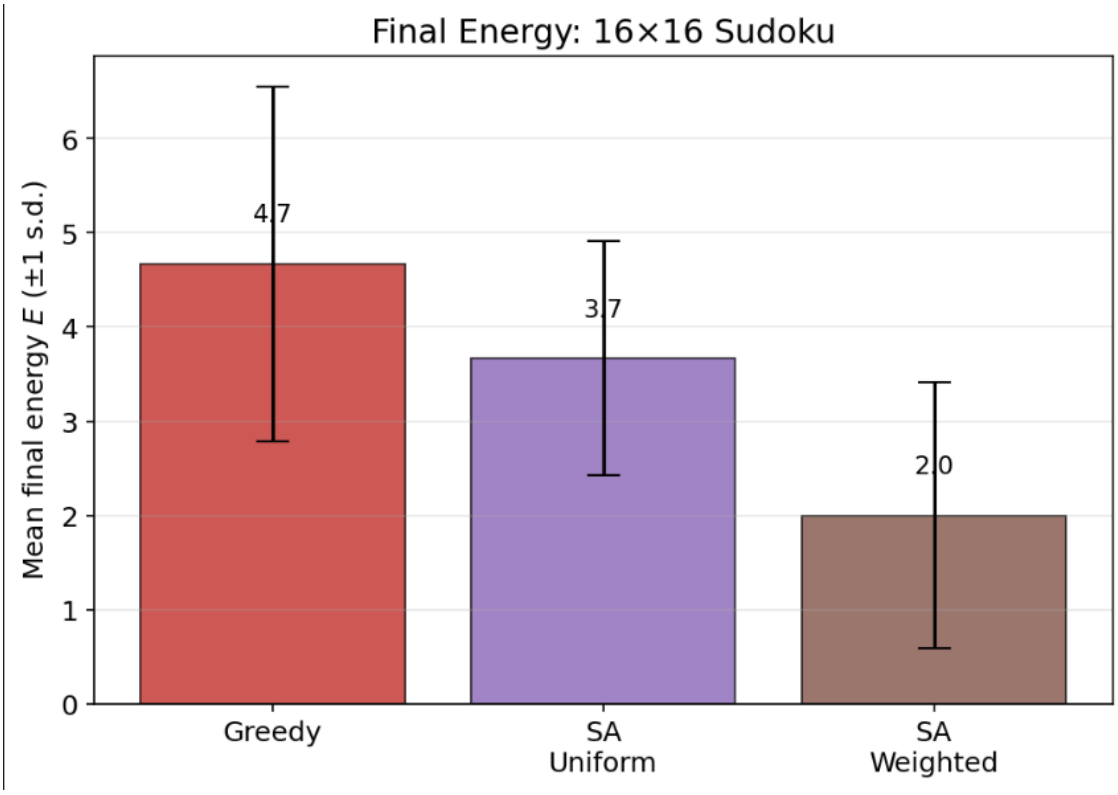


Figure 1: Mean final energy for greedy local search, uniform simulated annealing, and weighted simulated annealing on 16×16 Sudoku. Lower final energy indicates fewer remaining row and column violations. Error bars show one standard deviation.

Figure 1 shows a clear ordering among the three methods. Greedy local search has the highest mean final energy, approximately 4.7, which indicates that it is most likely to become trapped before reaching a valid solution. Uniform simulated annealing improves the mean final energy

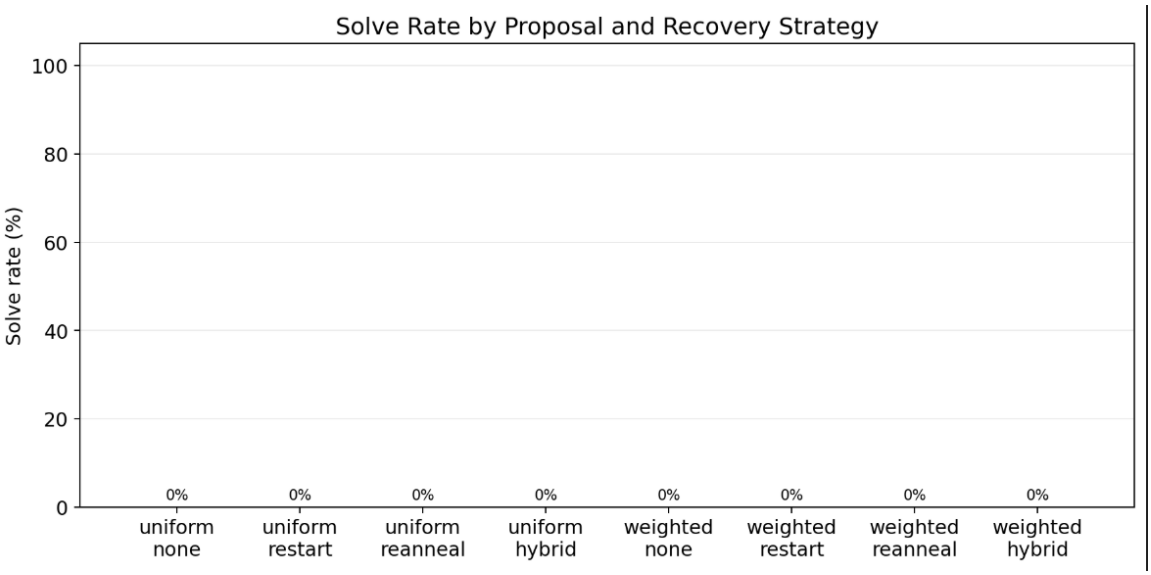
to approximately 3.7. This decrease supports the main purpose of simulated annealing: allowing occasional uphill moves helps the algorithm escape local minima that would trap a purely greedy method.

Weighted simulated annealing performs best in this comparison, with mean final energy approximately 2.0. This is less than half of the greedy method's final energy and noticeably lower than the uniform simulated annealing result. The improvement suggests that the proposal rule matters in addition to the acceptance rule. Simulated annealing gives the search a mechanism for escaping local minima, while the weighted proposal helps the search spend more time modifying cells that are actually involved in row and column conflicts.

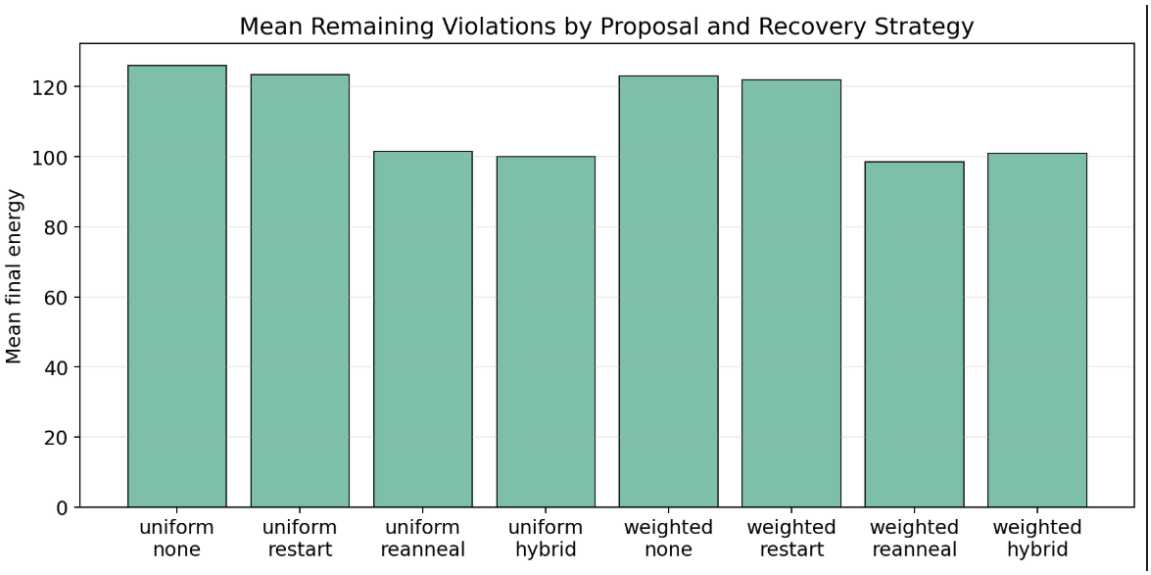
The standard deviation bars show that all three methods have run-to-run variability. This is expected because each method begins from a randomized completion of the subgrids and then follows a random sequence of swaps. However, the overall trend is still meaningful. Overall, the baseline experiment shows that each added algorithmic feature improves the final board quality. Allowing uphill moves improves on greedy search, and adding violation-weighted proposals improves on uniform simulated annealing. In terms of mean final energy, the methods therefore rank from weakest to strongest as greedy local search, uniform simulated annealing, and weighted simulated annealing.

3.3 Proposal and Recovery Comparison

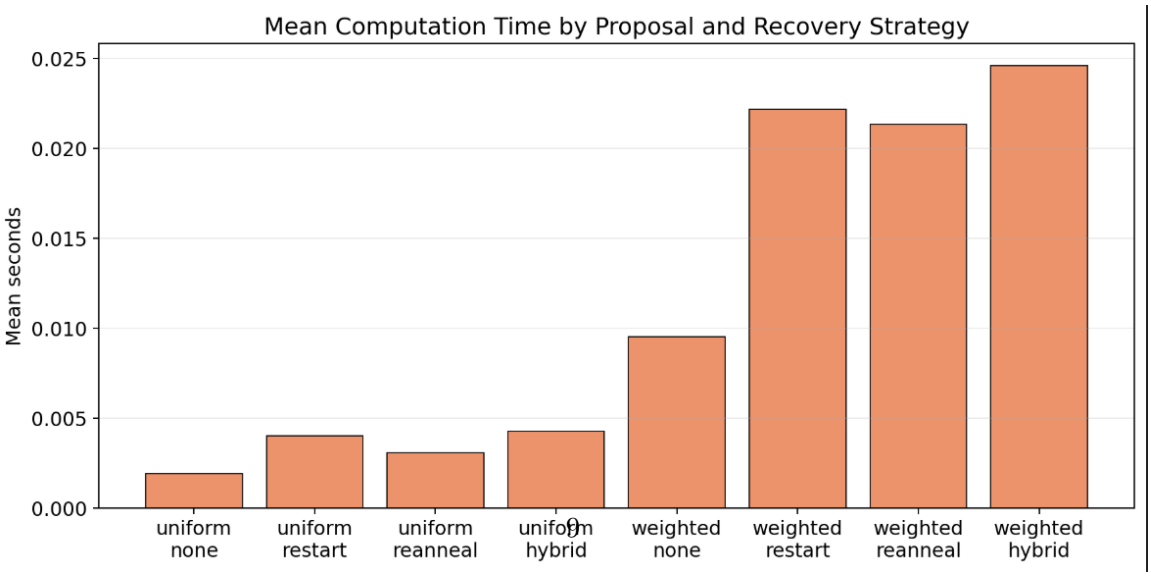
The next experiment compares proposal rules and recovery strategies together. This directly addresses the question of what happens when a simulated annealing run ends with positive energy. A positive-energy board is not a valid solution, but it may contain useful partial progress. The recovery strategies determine whether that progress is discarded, preserved, or combined with a later restart.



(a) Solve rate



(b) Mean final energy



(c) Mean computation time

To evaluate the interaction between proposal rule and recovery strategy, three performance measures were computed for each method: exact solve rate, mean final energy, and mean computation time, as shown in Figure 2. Solve rate measures how often the algorithm reached a valid solution, final energy measures how close the remaining failed runs were to a solution, and computation time measures the practical cost of each strategy. Considering these three measures together is important because solve rate alone can hide meaningful differences between methods when exact success is rare.

Panel 2a reports exact solve rate for combinations of proposal rule and recovery strategy. In the plotted comparison, all solve rates are 0%. This is a strict result: none of the methods reached energy zero under those specific settings. However, this does not mean all methods performed equally. Solve rate is a binary metric. A run ending with one remaining violation and a run ending with one hundred remaining violations are both counted as failures.

Panel 2b gives a more detailed view of performance. The uniform no-recovery condition has the highest remaining energy, approximately 126. Uniform restart is slightly lower, approximately 123, but the improvement is modest. Uniform re-annealing and uniform hybrid recovery are substantially better, ending near 101 and 100, respectively. This shows that recovery methods can matter even when exact solve rate remains zero.

The same general pattern appears in the weighted conditions. Weighted no-recovery and weighted restart remain high, around 123 and 122. Weighted re-annealing is lower, approximately 99, making it one of the best-performing methods in terms of final energy. Weighted hybrid is close, around 101. Although weighted hybrid does not dominate every other condition in this particular plotted run, it remains the most flexible design because it combines re-annealing and restart.

The strongest conclusion from Panel 2b is that re-annealing is more valuable than simply restarting when the current board is already close to a solution. Restart throws away all current structure. Re-annealing keeps the best board and raises the temperature, allowing temporary damage in order to escape a local minimum. In an energy landscape with many near-solutions, this is an important advantage.

Panel 2c shows the computational cost of each proposal-recovery combination. Uniform no-recovery is fastest, taking roughly 0.002 seconds on average in the plotted run. Uniform restart, re-annealing, and hybrid recovery remain below about 0.005 seconds. The weighted methods are slower. Weighted no-recovery is around 0.010 seconds, while weighted restart, weighted re-annealing, and weighted hybrid are roughly between 0.021 and 0.025 seconds.

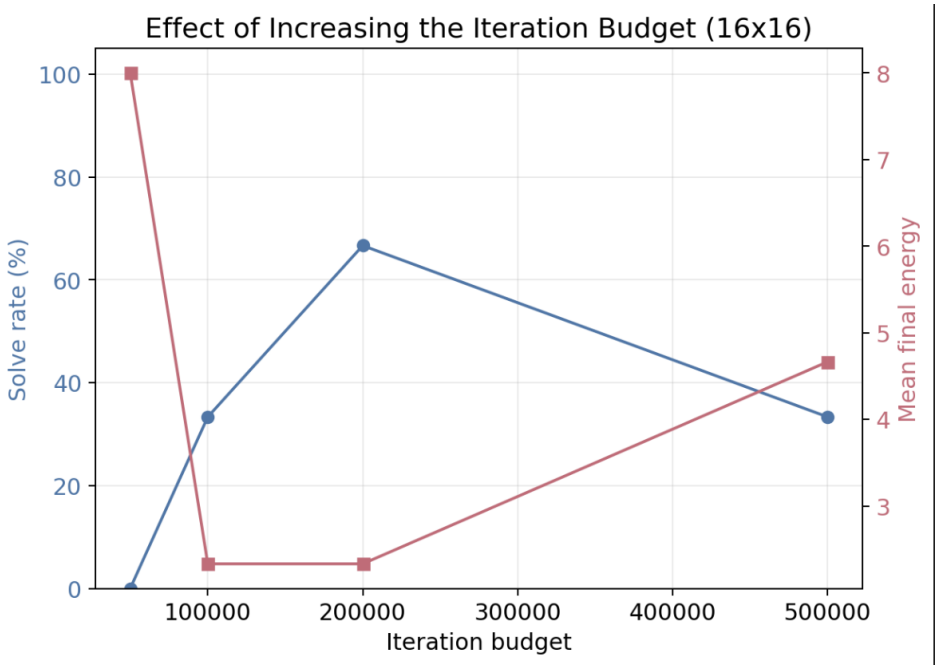
This runtime pattern is expected. Weighted proposals require additional computation because the algorithm evaluates row and column violation information before selecting a move. Recovery strategies also increase time because they perform additional annealing attempts. Weighted hybrid is the most expensive condition in the plot, but that cost reflects its more complex search behavior.

The practical interpretation is that algorithm quality cannot be judged only by final energy. A method that reduces energy but takes much longer may or may not be worthwhile depending on the application. For generalized Sudoku, the weighted methods are slower, but still computationally reasonable for the tested board sizes. If the goal is exact solution of difficult puzzles, the extra cost may be justified. If the goal is only to quickly reduce violations, uniform proposals may be acceptable.

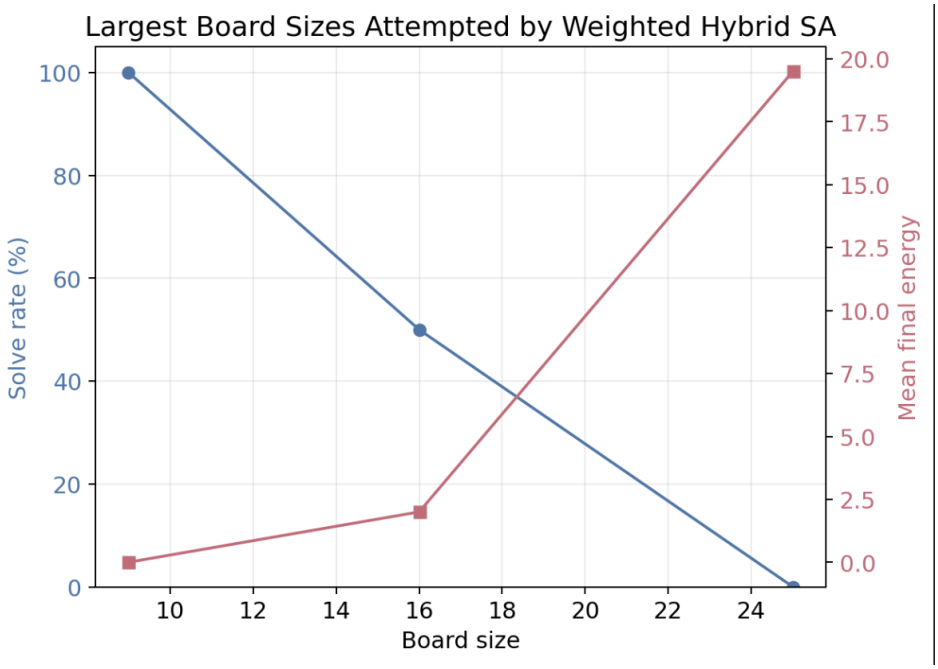
3.4 Iteration Budget and Board Size

To study how the algorithm scales, two additional experiments were performed, as shown in Figure 3. The first experiment varied the iteration budget on 16×16 puzzles in order to test whether longer runs improved performance. The second experiment varied the board size from 9×9 to

16×16 and 25×25 in order to test how large a puzzle the algorithm could solve under the same general settings.



(a) Effect of iteration budget on 16×16 puzzles



(b) Effect of board size

Figure 3: Grouped scaling results for weighted hybrid simulated annealing. The first panel studies whether increasing the iteration budget improves 16×16 performance. The second panel studies how performance changes as the board size increases from 9×9 to 16×16 and 25×25.

Panel 3a examines the effect of increasing the iteration budget for 16×16 puzzles. The smallest budget, 50,000 iterations, has 0% solve rate and mean final energy around 8. Increasing the budget to 100,000 iterations improves the solve rate to about 33% and reduces mean final energy to about 2. At 200,000 iterations, the solve rate rises to about 67%, while mean final energy remains close to 2. At 500,000 iterations, the solve rate decreases to about 33%, and mean final energy rises to approximately 4.5.

The algorithm shows an overall improvement as the iteration budget increases from 50,000 to 100,000 and then to 200,000. This trend suggests that the search needs enough time to escape early local minima and settle into lower-energy regions. With only 50,000 iterations, the process may stop before simulated annealing has enough opportunity to use uphill moves effectively. By 100,000 and 200,000 iterations, the solve rate increases and the mean final energy decreases, which indicates that the additional search time is useful.

However, there is a drop at 500,000 iterations. This behavior does not mean that running the algorithm longer is generally worse. Theoretically, giving simulated annealing more iterations should usually improve the chance of finding a solution, as long as the cooling schedule and recovery strategy are chosen appropriately. The result here shows the opposite pattern only for this plotted experiment. The main reason is the small number of trials. With few trials, random initialization and random move sequences can have a large effect on the reported percentages. As a result, a few unlucky runs at 500,000 iterations can make that budget appear worse than 200,000 iterations, even though the larger budget should be expected to perform better on average over many more trials.

The impact of Panel 3b is that scaling matters: a method that works well for 9×9 puzzles may require significantly more tuning for 16×16 and 25×25 puzzles because larger boards have more cells, more possible swaps, and more interacting row and column constraints. As the search landscape becomes more complex, recovery strategies and proposal design become more important. For 9×9 boards, the weighted hybrid method achieves 100% solve rate and mean final energy 0, confirming that the algorithm can solve standard-size Sudoku instances under the tested settings. For 16×16 boards, the solve rate drops to about 50%, and mean final energy rises to about 2, which indicates partial success: the algorithm often gets close to a valid solution, but it does not solve every instance within the same budget. For 25×25 boards, the solve rate drops to 0%, and mean final energy rises to about 20. This is a substantial increase in difficulty. The algorithm still reduces conflicts, but the tested budget is not enough to reach exact solutions. Therefore, the 25×25 result should be interpreted as a limitation of the current implementation and parameter choices, not as evidence that simulated annealing cannot solve such boards.

4 Discussion

The main impact of the experiments is that generalized Sudoku should be evaluated with more than one metric. Exact solve rate is important because a positive-energy board is not a valid solution. However, solve rate alone does not describe how close the algorithm came to solving the puzzle. The proposal-recovery comparison makes this clear: all exact solve rates are zero, but the final energies differ substantially. This means that the methods are not equivalent, even though the solve-rate plot alone would make them appear equivalent.

Positive-energy outputs are especially important in simulated annealing because they often represent local minima. A run that ends at energy 100 is not successful, but it may still contain useful structure. A run that ends at energy 2 is even more informative: it is almost valid, and a good recovery strategy may be able to repair it. This is why re-annealing and hybrid recovery are

meaningful. They treat a failed run as a partial result rather than a discarded result.

The weighted proposal rule also has an important impact. Uniform proposals are simple and theoretically clean, but they do not use any information about where conflicts occur. Weighted proposals focus the search on conflicted rows, columns, and cells. The baseline comparison shows that weighted simulated annealing produced a lower mean final energy than uniform simulated annealing and greedy search. This suggests that conflict-directed proposals are useful for generalized Sudoku.

At the same time, weighted proposals introduce a theoretical complication. The standard Metropolis acceptance probability assumes a symmetric proposal rule. The weighted proposal is not generally symmetric because the probability of proposing a move can change after the move is made. Therefore the weighted method should be described as a heuristic unless the Metropolis-Hastings correction ratio is included. This distinction is important because it separates practical performance from theoretical Markov chain guarantees.

Recovery strategy affects how the algorithm responds to local minima. Full restart is useful when the current board is poor, but it can waste progress. Re-annealing is useful when the current board is close to a solution because it preserves the board while restoring the ability to accept uphill moves. Hybrid recovery is conceptually strongest because it combines both approaches. It can try to rescue a near-solution through re-annealing, but it can still restart if the current region appears unproductive.

The board-size experiment shows that generalized Sudoku becomes substantially harder as size increases. The algorithm solves 9×9 boards reliably, has partial success on 16×16, and struggles on 25×25. This pattern is expected because the number of cells and possible configurations grows quickly. The 25×25 case is the most interesting future target because it is large enough to expose the limitations of the current implementation.

Parallelization is possible, but updating different blocks simultaneously is not straightforward. A swap inside one block affects row and column conflicts, and those rows and columns pass through other blocks. Therefore simultaneous block updates may interfere with one another. The safest parallel approach is to run independent annealing chains in parallel. Each chain starts from a different random initialization and follows its own random path. The computation stops when one chain finds a solution. This does not change the Markov chain itself; it simply explores more trajectories at the same time.

Several limitations remain. First, the number of trials is limited. Since simulated annealing is random, small trial counts can be misleading, especially for solve rate, where one additional success can noticeably change the reported percentage. A stronger experiment would use many more trials and report standard deviations or confidence intervals. Second, the weighted proposal rule is heuristic because the current implementation does not include the full Metropolis-Hastings proposal ratio. A more theoretically justified version would compute $q(X, Y)$ and $q(Y, X)$ and use the corrected acceptance probability. Third, the 25×25 case remains difficult. The current implementation reduces conflicts on larger boards, but it does not solve them reliably under the tested settings. Future work should therefore combine larger budgets, more trials, improved weighted proposals, and parallel independent annealing chains.

5 Conclusion

This project models generalized Sudoku as a finite stochastic optimization problem. The state-space restriction preserves fixed clues and block validity, so the energy function only counts row and column violations. Simulated annealing is a natural method because it can accept uphill moves

and escape local minima.

The experiments show that proposal rules and recovery strategies matter. Weighted proposals guide the search toward conflicted regions and produce lower mean final energy than uniform proposals and greedy local search in the baseline comparison. Re-annealing and hybrid recovery are meaningful responses to positive-energy outputs because they preserve useful partial progress. The results also show that larger puzzles require substantially more computation. The method works best on smaller boards, has partial success on 16×16, and struggles on 25×25 within the tested budget.

The most important lesson is that exact solve rate should not be reported alone. For generalized Sudoku, final energy, runtime, board size, and recovery behavior all reveal important information about the search process. Simulated annealing is useful not only because it sometimes solves the puzzle, but also because its failures provide structured information about the energy landscape.

Use of AI

ChatGPT was used to help organize the LaTeX structure, revise wording and check grammar. It was also used to help debug code issues and support portions of the coding and computational optimization workflow. AI served only as a support resource for revision and implementation-related tasks, not as a source of the project's primary content or analysis.

References

[1] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

[2] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[3] B. Hajek. Cooling schedules for optimal annealing. *Mathematics of Operations Research*, 13(2):311–329, 1988.

[4] T. Yato and T. Seta. Complexity and completeness of finding another solution and its application to puzzles. *IEICE Transactions on Fundamentals*, E86-A(5):1052–1060, 2003.

[5] R. Lewis. Metaheuristics can solve Sudoku puzzles. *Journal of Heuristics*, 13:387–401, 2007.